Consistent Hashing

add a server, move only a small share of the keys

Phase 3 videos on YouTube

In one line: with `hash(key) mod n`, changing n moves almost every key; consistent hashing puts servers and keys on a ring and gives each key to the next server clockwise, so adding or removing a server moves only about 1/n of the keys, and virtual nodes keep the load even.

1 The problem with modulo

You spread cache keys across four servers with `server = hash(key) mod 4`. It works well, until you add a fifth server. Now `mod 5` sends most keys to a different server than before. For a cache that's almost a cold restart: the hit rate collapses, and every miss goes to the database.

The reason: a key's server depends on the **total number** of servers. Going from 4 to 5 moves about 80% of the keys; in general, going from n to n + 1 moves about n/(n + 1) of them. What we want is that only the keys the new server should own move: about 1/(n + 1).

key hash	hash mod 4	hash mod 5
17	server 1	**server 2**
26	server 2	**server 1**
39	server 3	**server 4**
45	server 1	**server 0**
60	server 0	**server 0**

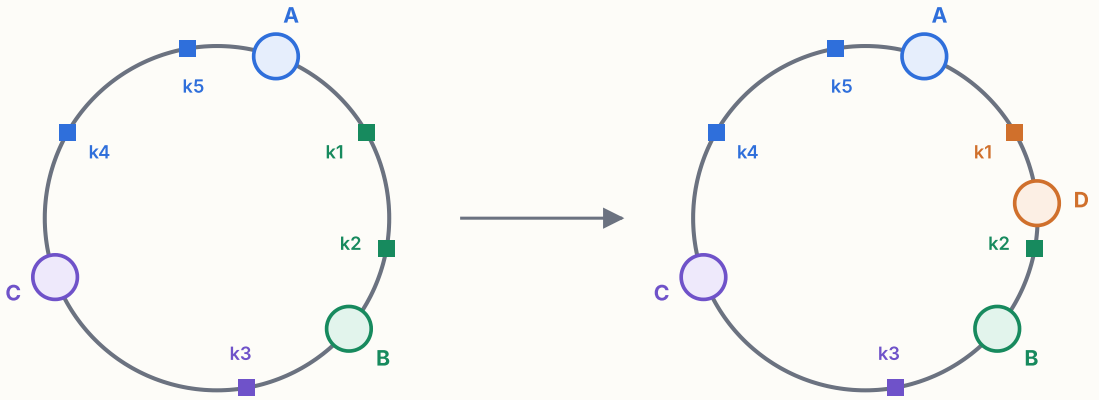
4 → 5 servers with modulo:
about 80% of keys change server
a cache becomes nearly cold;
the database takes the misses

Figure 1. With modulo hashing, changing the server count changes the answer for most keys.

2 The ring

Take the output range of a hash function (say 0 to $2^{32} - 1$) and bend it into a circle. Hash each **server** (by name or address) to a position on the ring. To find a key's server, hash the **key** to a position, then walk **clockwise** to the first server you meet. That server owns the key.

Now add a server. It lands at one point and takes over only the keys between it and the previous server counter-clockwise. Every other key stays where it was. Remove a server, and only its keys move, to the next server clockwise. On average only **1/n** of the keys move: the minimum possible.



each key belongs to the next server clockwise

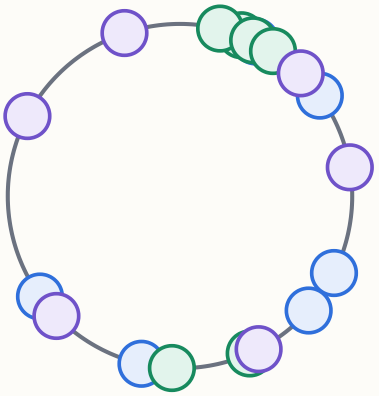
add D: only k1 moves (from B to D)

Figure 2. Left: three servers on the ring, each key owned by the next server clockwise. Right: adding D takes over only the arc between B's predecessor and D.

3 Virtual nodes

With only a few servers, their random positions can be very uneven: one server might own a huge arc, another a tiny one. And when a server leaves, all its keys land on just one neighbour, which may then be overloaded.

Virtual nodes fix both. Each physical server is hashed to **many** points on the ring, say 100 to 200 (hash `A#1`, `A#2`, ...). The many small arcs average out, so each server gets close to its fair share. A departing server's keys scatter across many servers. And **weights** become easy: give a bigger server more virtual nodes.



3 servers × 6 virtual nodes

Without virtual nodes, 3 random points can give one server 60% of the ring and another 10%.

With 100–200 virtual nodes per server:

- the arcs average out, so load is even
- a failed server's keys spread over many servers
- weights are easy: a bigger server gets more points

each physical server appears at many positions

Figure 3. Virtual nodes: each server owns many small arcs instead of one big one.

WORKED EXAMPLE: ADDING A MEMCACHED NODE FOR THE HOLIDAY PEAK

A Memcached cluster of four nodes uses a client library with consistent hashing (the classic one is **ketama**). You add a fifth node for the holiday peak. Only about 20% of keys move to the new node; the other 80% keep hitting the same servers. The database sees a small bump in misses, not a stampede. With plain modulo, about 80% of keys would have moved instead.

The same idea places data in **DynamoDB** and **Cassandra** (each node owns token ranges on a ring) and powers hash-based load balancing such as Envoy's ring hash and Maglev.

4 Alternatives

Method	How	Good	Catch
Ring + virtual nodes	next server clockwise	minimal movement, weights	needs many vnodes for even load; ring memory
Rendezvous hashing (highest random weight)	score every server with hash(key, server), pick the highest	no ring, very even, minimal movement	O(n) per lookup, slower with many servers
Jump consistent hash	a few lines of arithmetic map a key to a bucket 0..n−1	tiny, fast, near-perfect balance, no memory	servers can only be added or removed at the end

COMMON MISTAKES

- Using modulo hashing for a cache or shard map that will ever change size.
- One point per server, then wondering why one node holds 40% of the data.
- Different clients using different hash functions or server lists, so they disagree about where a key lives.
- Forgetting hot keys: consistent hashing spreads keys evenly, not traffic, so one very popular key still lands on one server (lesson 26).

5 Key terms

Modulo hashing: hash(key) mod n; moves most keys when n changes

Owner: the first server clockwise from the key

Ketama: classic consistent hashing for Memcached clients

Jump hash: fast arithmetic consistent hash for numbered buckets

Hash ring: the hash range bent into a circle

Virtual node: one of many ring positions for a server

Rendezvous hashing: pick the server with the highest hash(key, server)

Token range: the arc of the ring a Cassandra node owns

6 Interview questions

Q1. Explain consistent hashing. BASIC

Servers and keys are hashed onto a ring, and each key belongs to the next server clockwise, so adding or removing a server only moves about 1/n of the keys, instead of nearly all of them as with modulo.

Q2. Why use virtual nodes? BASIC

They give each server many positions on the ring, which evens out the load, spreads a failed server's keys across many others instead of one neighbour, and allows weights for bigger servers.

Q3. You go from 4 to 5 cache nodes. How many keys move with modulo, and with consistent hashing? SCENARIO

With modulo, about 4/5, 80% of keys. With consistent hashing, about 1/5, 20%: just the share the new node should own.

Q4. Where is consistent hashing used in real systems? SCENARIO

Distributed caches (Memcached clients with ketama, Redis client-side sharding), databases such as Cassandra and DynamoDB that assign token ranges to nodes, and load balancers that need stable key-to-backend mapping (Envoy ring hash, Maglev).

Q5. Consistent hashing gives each node the same number of keys, yet one node is overloaded. Why? TRICKY

Keys are spread evenly, but traffic isn't: a hot key (a celebrity's profile) sends all its requests to one node. Fix with replication of hot keys, local caching, or key splitting.

Q6. When would you pick rendezvous or jump hashing instead of a ring? TRICKY

Rendezvous when the server list is small and you want very even balance without vnodes; jump hash when buckets are numbered and only grow or shrink at the end, such as shards 0..n−1, because it needs no memory and is very fast.

Next: SD-G-006 · IP Hash and Sticky Sessions backendroadmap.com